

Informe trabajo Programación Mouvi

1. Pasos realizados:

- Creamos el repo TodoApp-Group777
- Agregamos a los otros miembros del trabajo como colaboradores.
- Clonamos el repositorio en nuestros computadores.
- Creamos los archivos iniciales: main.py, task_model.py y .gitignore.
- Hacimos commit y push de la estructura base.
- Cada estudiante crea su propia rama: Serge, Olmos, Rodriguez.
- Serge añadió funcionalidad para marcar tareas como completadas.
- Olmos añadió funcionalidad para eliminar tareas y actualizo README.md.
- Rodriguez simuló un cambio conflictivo en task_model.py y lo resolví.
- Creamos la rama Group777 y fuimos fusionando las ramas individuales.
- Resolvimos conflictos y fuimos con la rama main mediante Pull Request.
- Eliminamos ramas individuales y la de grupo.

2. Comandos Git:

- `git clone <URL>`: Clonar Repositorio
- `git checkout -b <rama>`: Crear una nueva rama.
- `git add <archivo>`: Añadir cambios al área de preparación
- `git commit -m "mensaje"`: Confirmar cambios.
- `git push origin <rama>`: Subir cambios al repositorio.
- `git merge <rama>`: Fusionar ramas.
- `git push origin --delete <rama>`: Eliminar rama.

3. Conflictos:

- Conflicto en `task_model.py` entre atributos `ts_completed` / `ts_done` y métodos `mark-as-complete` / `set-done`.
- También en `main.py` debido a cambios simultáneos en el flujo principal.
- La solución fue que combinamos cambios, manteniendo `ts_completed` como atributo principal y uniendo la lógica de los métodos.

4. Contribuciones:

- Serge: Añadido mark-as-complete en task_model.py y actualizo main.py.
- Olmos: Añadido delete-task en task_model.py y actualizo README.md.
- Rodriguez: Simulo conflicto con set-done y remove-task, y resolvo conflictos en task_model.py y main.py.

5. Reflexiones:

- Aprendimos a coordinar ramas y resolver conflictos trabajando en equipo.
- Git es una herramienta potente para trabajo colaborativo.
- Los pull request ayudan a mantener la calidad del código y a evitar errores.

Preguntas de Reflexión

¿Cómo coordinar el trabajo en equipo para evitar conflictos innecesarios?

R/ Nos organizamos asignando tareas específicas a cada uno y revisando cambios antes de hacer merge.

¿Que aprendieron sobre la resolución de conflictos en GIT?

R/ Que es importante entender el código de todos para combinar cambios sin perder funcionalidad.

¿~~Que aprendieron~~ Por que es importante usar Pull Request en proyectos colaborativos?

R/ Permite revisión de código y evita que errores pasen a producción.

¿Cómo podría mejorarse el flujo de trabajo con herramientas como CI/CD?

R/ Automatizando pruebas y revisiones antes de integrar cambios a main.

Descripción historial commits

- 599bd21: Initial commit.
- e901522: Inicializar Proyecto To Do App
- 44153a3: (Origin/Olmos, Olmos): Añadir funcionalidad para eliminar y actualizar tareas.
- d567e2a: (Origin/Serge): Añadir funcionalidad para marcar tareas como completadas.
- 2d73615: (Origin/Rodriguez): Añadir set_done y remove_task en TaskModel.
- 7fc5703: Merge remotetracking branch 'origin/Rodriguez' into Group777.
- bb65cc2: (Origin/Group777): Resolver conflicto combinando métodos y atributos.
- ec14439: (HEAD --> main, origin/main, origin/HEAD): Merge pull request #1 from jmserge29/Group777.